

MAI (Mravel AI) — Giải thích toàn bộ theo từng luồng

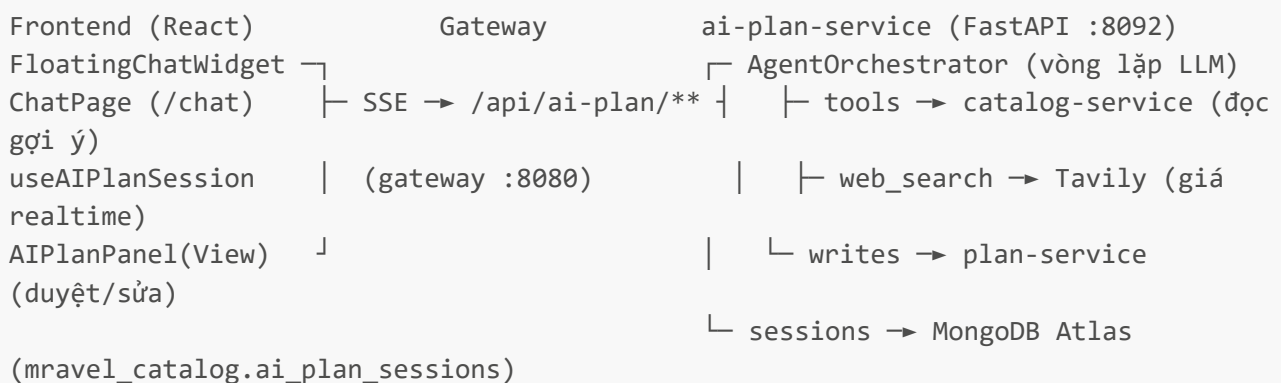
Tài liệu này giải thích AI trên website Mravel (backend `ai-plan-service` + frontend React): đi theo **từng luồng (flow)**, mỗi luồng nêu rõ **file nào làm gì** và **giải thích các đoạn code quan trọng**. Phần **cấu hình** ở cuối file.

Tài liệu bạn đọc; code/comment trong repo vẫn bằng tiếng Anh theo quy ước dự án. Tài liệu liên quan: `STORAGE.md` (chi tiết lưu trữ).

0. MAI là gì (tóm tắt)

MAI **không phải model tự train**. Nó là lớp **điều phối (orchestration)** quanh một LLM có sẵn (qua API OpenAI-compatible), dựa trên 4 trụ cột: **gọi công cụ (tool calling)** · **truy hồi dữ liệu (retrieval)** · **đầu ra có cấu trúc (structured output)** · **luồng duyệt (approval)**. Nguyên tắc: **draft-first** — AI chỉ *đề xuất*, không ghi vào plan thật cho tới khi người dùng bấm **Duyệt/Áp dụng**.

Sơ đồ tổng:



Luồng 1 — Khởi động & cấu hình

File liên quan

- `app/main.py` — tạo FastAPI app, gắn CORS, router, endpoint health.
- `app/config.py` — toàn bộ biến môi trường (LLM, Mongo, web search, CORS...).
- `app/api/dependencies.py` — "dây nối" Dependency Injection: tạo các singleton (llm, catalog client, plan client, store, orchestrator...).
- `app/llm/factory.py` — chọn LLM client: stub (không key) hay OpenAI-compatible.

Code quan trọng

`config.py` dùng pydantic-settings, đọc `.env`. Hai hàm quyết định hành vi:

```
def has_llm(self) -> bool: # rỗng key → chạy STUB (không gọi model)
    return bool(self.llm_api_key.strip())
```

```
def fallback_model_list(self) -> List[str]: # các model dự phòng khi model chính
    lỗi
    return [m.strip() for m in self.llm_fallback_models.split(",") if m.strip()]
```

factory.py — chọn cài đặt LLM theo key:

```
def build_llm_client(settings):
    if not settings.has_llm():
        return StubLLMClient() # regex + dựng nháp tất định, không
    cần model
    return OpenAILLMClient(api_key=..., base_url=..., model=...,
        fallback_models=...)
```

dependencies.py — bơm `max_tokens`, `web_base_url` vào orchestrator:

```
settings = get_settings()
return AgentOrchestrator(llm, catalog, composer, plan_client=plan_client,
    web_search=web_search,
    web_base_url=settings.public_web_base_url,
    max_tokens=settings.llm_max_tokens)
```

→ `llm_max_tokens=8000` rất quan trọng: kế hoạch dày (nhiều card trong 1 lệnh) cần nhiều token, để 2048 sẽ bị cắt cụt JSON.

Luồng 2 — Gửi tin nhắn (streaming SSE)

Đây là luồng người dùng dùng nhiều nhất.

File liên quan

- `app/api/ai_plan.py` — endpoint `POST /sessions/{id}/messages/stream`; dò plan id; gate quyền; chọn nhánh edit hay dựng-nháp; gói SSE.
- `app/agent/orchestrator.py` — `plan_stream` / `edit_stream`: vòng lặp agent.
- Frontend `useAIPlanSession.js` — đọc luồng SSE, cập nhật state.
- `app/api/schemas.py` — model request/response.

Code quan trọng — `stream_message` quyết định nhánh:

```
session.messages.append(ChatMessage(role=USER, content=body.content));
store.save(session)

# Nếu user nhắc tới một plan (vd dán /plans/43) → bind phiên vào plan đó để CHỈNH
SỬA
detected_plan_id = _extract_plan_id(body.content)
if detected_plan_id is not None and detected_plan_id != session.plan_id:
    try:
```

```

        await _assert_can_edit(plan_client, user.raw_token, detected_plan_id) #
        kiểm tra quyền
    except ForbiddenError as exc:
        return StreamingResponse(_one_shot_assistant_stream(...exc.message...)) #
        từ chối mềm
        session.plan_id = detected_plan_id; store.save(session)

if session.plan_id is not None:
    return StreamingResponse(_consume_edit_stream(...)) # → Luồng 5 (sửa plan)
# ngược lại → trích constraints rồi _consume_agent_stream → Luồng 3 (dừng nháp)

```

`_sse_event` chuyển mỗi sự kiện thành khung SSE (`event: + data:`). Bộ sự kiện: `session`, `thinking`, `tool_call`, `tool_result`, `assistant_message`, `draft_ready`, `edit_proposal`, `constraints_updated`, `error`, `done`.

Phía FE, `useAIPlanSession.js` đọc từng sự kiện trong `runStream`:

```

} else if (event === "tool_call") { setActivity(prev => [...prev, {text: `→
${labelForTool(...)}`}]); }
} else if (event === "draft_ready") { setDraft(data.draft); }
} else if (event === "edit_proposal") { setPendingEdits(data.operations); }
} else if (event === "session") { if (data.plan_id !== null)
setBoundPlanId(data.plan_id); } // bật edit-mode động

```

Luồng 3 — Dừng nháp kế hoạch mới (plan mode)

File liên quan

- `app/services/constraint_extractor.py` — trích điểm đến/ngày/người/ngân sách (lai regex + LLM).
- `app/agent/orchestrator.py` — `plan_stream` + hằng `_SYSTEM_PROMPT` (luật + mật độ).
- `app/agent/tools.py` — định nghĩa tool + `ToolDispatcher` thực thi.
- `app/clients/catalog_client.py` — gọi REST catalog-service.
- `app/clients/web_search_client.py` — Tavily.
- `app/services/geo.py` — tính khoảng cách/ thời gian (`route_legs`).
- `app/services/draft_composer.py` — dựng nháp tắt định (chế độ stub / fallback).
- `app/models/session.py` — `Constraints`, `PlanDraft`, `DraftActivity`.

Code quan trọng — vòng lặp agent (rút gọn từ `_run_agent_loop_stream`):

```

for iteration in range(MAX_ITERATIONS): # tối đa 10 vòng
    turn = await self._llm.chat_with_tools(_SYSTEM_PROMPT, messages, tools,
                                           max_tokens=self._max_tokens) # 8000
    if turn.stop_reason != "tool_use": # model trả lời text → kết thúc
        lượt
        yield {"event": "assistant_message", "text": turn.text}; return
    for tu in turn.tool_uses: # chạy từng tool model yêu cầu

```

```

        if tu.name == "set_constraints": working =
apply_set_constraints(tu.arguments, working)
        elif tu.name == "finalize_draft": finalized =
parse_finalize_draft(tu.arguments, working)
        else: result = await dispatcher.run(tu.name, tu.arguments) #
search/web/route
        messages.append(_tool_result(tu.id, result))
    if finalized: yield {"event":"draft_ready","draft":finalized}; return

```

Ý tưởng: model **nghĩ** → **gọi tool** → **đọc kết quả** → **nghĩ tiếp** cho tới khi gọi **finalize_draft** (tool kết thúc) để nộp nháp nhiều ngày.

tools.py — **ToolDispatcher.run** ánh xạ tool → client. Ví dụ **search_places** ưu tiên endpoint **faceted theo slug** (chống mất dấu tiếng Việt):

```

if destination:                                # "Đà Nẵng" → slug "da-nang"
(accent-proof)
    _add(await self._catalog.search_places_faceted(location_slug(destination),
...))
if query:                                       # truy vấn tự do, GIỮ dấu
    _add(await self._catalog.search_places(query, ...))

```

Mỗi item gợi ý được gắn **mravel_url** (**/hotels/<slug>**, **/place/<slug>**...) để chat có link bấm được:

```

def catalog_web_url(kind, slug, base_url=""):
    path = _CATALOG_PATHS.get(kind.upper()) # HOTEL→/hotels, PLACE→/place, ...
    return f"{base_url.rstrip('/')}{path}/{slug}" if slug and path else None

```

constraint_extractor.py — lại: regex chạy trước (luôn có baseline), LLM tinh chỉnh sau:

```

baseline = _normalize(await _STUB_EXTRACTOR.extract_constraints(msg, prior_dict),
prior)
if isinstance(self._llm, StubLLMClient): return baseline # không có model thật →
dừng ở regex
llm_result = await self._llm.extract_constraints(msg, baseline_dict) # tinh chỉnh

```

Luồng 4 — Hỏi đáp / khám phá + link Mravel

Người dùng hỏi "khách sạn nào đặt được trên web?", "cho link địa điểm Đà Nẵng".

File liên quan: **orchestrator.py** (mục "Answering questions & sharing links" trong **_SYSTEM_PROMPT** / "kind 2" trong **_EDIT_SYSTEM_PROMPT**), **tools.py** (**ToolDispatcher** + **_empty_location_fallback**), **catalog_client.py**.

Code quan trọng — khi search theo location ra rỗng, nói rộng để gợi ý nơi gần:

```

async def _empty_location_fallback(self, kind, location):
    rows = await self._catalog.search_hotels(None, ...) # bỏ location → lấy
    tất cả
    return {"items": [], "no_results_for_location": location,
           "available_on_mravel": [_summarise_hotel(r, self._web_base_url) for r
    in rows],
           "note": "Mravel chưa có ... 'available_on_mravel' là lựa chọn đang có
    (có thể ở thành phố lân cận)"}

```

Prompt ép: search catalog **trước**, trả **link mravel_url**; rồi thì gợi ý **available_on_mravel**; cùng đường mới **web_search** (gắn nhãn "ngoài Mravel"). **Không bịa quán, không dán link ngoài từ trí nhớ.**

Luồng 5 — Chỉnh sửa plan (edit mode) + mật độ/đầy field

Khi phiên đã gắn **plan_id** (do user nhắc tới plan).

File liên quan

- `app/agent/orchestrator.py` — `edit_stream` + `_EDIT_SYSTEM_PROMPT` (gồm mục "BE DENSE"
 - "Fill EVERY field").
- `app/agent/edits.py` — model `EditOperation`, `parse_operations` (validate), `board_summary`.
- `app/services/edit_service.py` — biến op → lệnh ghi plan-service.
- `app/clients/plan_client.py` — REST plan-service.

Code quan trọng — `edit_stream` đọc board + **chốt quyền** rồi mới cho đề xuất:

```

board = await self._plan_client.get_board(bearer_token, plan_id)
role = str(board.get("myRole") or "").upper()
if role not in ("OWNER", "EDITOR"):
    yield {"event": "assistant_message", "text": "Bạn chỉ có quyền xem ... không thể
    đề xuất sửa."}
    return # không dựng đề xuất cho viewer
summary = board_summary(board) # tóm tắt board: list_id/card_id để
model nhắm đúng

```

`edits.py` — `parse_operations` **loại op sai/ảo** (id không có trên board → bỏ):

```

if op not in OP_TYPES: continue # op lạ → bỏ
if board is not None and not _ids_exist(model, list_ids, card_ids): continue # id
ảo → bỏ

```

`edit_service.py::_card_body` — **lắp đầy field** vào `activityDataJson` (giống Excel + giống luồng duyệt), kèm description 📌 / 💡 / 📄:

```

activity_data = {k:v for k,v in {
    "reason":op.reason, "locationName":op.location_name, "address":op.address,
    "note":op.note, "routeHint":op.route_hint, "distanceText":op.distance_text,
    "transportMode":op.transport_mode, "ticketPrice":op.unit_price_vnd,
    "ticketCount":op.quantity,
}.items() if v}
if op.recommendation: activity_data["recommendation"] = {...} # giữ link/ảnh
catalog

```

Mật độ (yêu cầu của bạn): `_EDIT_SYSTEM_PROMPT` có mục bắt buộc — khi dựng/điền cả chuyến, làm trong **1 lệnh `propose_plan_edits`** phủ mọi ngày, **mỗi ngày ≥ 8 hoạt động** theo chuỗi *vệ sinh* → *ăn sáng* → *cafe* → *tham quan* → *ăn trưa* → *ngủ trưa* → *tham quan/khu vui chơi* → *ăn tối* → về, **mỗi card lấp đầy** giờ/địa chỉ/ghi chú/lộ trình/khoảng cách/phương tiện/chi phí/recommendation.

Lưu ý: model free `gpt-oss-120b` khá yếu — để mật độ ổn định nên đổi `LLM_MODEL` sang model mạnh hơn (xem Cấu hình). Và `add_day` chưa trả id ngay trong cùng lệnh, nên dựng từ plan 1 ngày cần 2 lượt (thêm ngày → rồi điền).

Người dùng bấm **Áp dụng** → `POST /apply-edits` → `EditService.apply` (kiểm quyền lại) → gọi lệnh board plan-service: `update_card` / `create_card` / `delete_card` / `move_card` / `rename_list` / `delete_list` / `add_day` / `update_plan`.

Luồng 6 — Duyệt nháp thành plan thật (approval)

File liên quan: `app/services/approval_service.py`, `app/clients/plan_client.py`.

Code quan trọng — tạo plan rồi gắn card vào các ngày plan-service **tự seed**, có **idempotency key** để bấm lại không nhân đôi:

```

plan = await self._plan_client.create_plan(bearer_token, plan_body)
board = await self._plan_client.get_board(bearer_token, plan_id) # đọc các
ngày đã seed
for index, day in enumerate(draft.days):
    list_id = _ordered_day_lists(board)[index]["id"]
    for position, activity in enumerate(day.activities):
        card_idem = _idem_key(f"plan:{plan_id}:card:{day.day_index}:{position}")
        await self._plan_client.create_card(bearer_token, plan_id, list_id,
                                            _activity_to_card_body(activity,
draft.travelers),
                                            idempotency_key=card_idem)

```

`_activity_to_card_body` là "khuôn mẫu" đầy đủ field (address/note/route/recommendation...) mà luồng edit (`_card_body`) đã được làm cho giống.

Luồng 7 — Lưu trữ phiên (MongoDB)

File liên quan: `app/services/session_store.py` (MongoSessionStore / FileBackedSessionStore / InMemorySessionStore + factory), `app/config.py`, `app/models/session.py` (PlanSession).

Code quan trọng — mỗi phiên là 1 document, lưu = **1 upsert** (không ghi lại cả file):

```
def save(self, session):
    session.updated_at = datetime.now(timezone.utc)
    self._col.replace_one({"_id": session.session_id}, self._to_doc(session),
upsert=True)
def list_by_user(self, user_id):
    # cho danh sách lịch sử
    return [self._from_doc(d) for d in
self._col.find({"user_id":user_id}).sort("updated_at", DESCENDING)]
```

Factory chọn backend theo `SESSION_STORE_BACKEND` (mongo mặc định), **fallback** file/ memory nếu Mongo không kết nối được. Chi tiết + cách xem Compass: `STORAGE.md`. Dữ liệu nằm ở **Atlas** → **DB mravel_catalog** → **collection ai_plan_sessions**.

Luồng 8 — Frontend (giao diện chat)

File liên quan

- `src/features/aiPlan/hooks/useAIPlanSession.js` — "bộ não" FE: mở phiên, đọc SSE, giữ state (messages/draft/pendingEdits), lịch sử (sessions/refreshSessions/switchSession), boundPlanId (bật edit-mode động).
- `src/features/aiPlan/components/AIPlanPanel.jsx` — UI chat (AIPlanPanelView): render Markdown bằng `marked` (link bấm được), overlay lịch sử, nút "Hội thoại mới".
- `src/features/chat/components/FloatingChatWidget.jsx` — ghim **MAI** ở mọi trang; **giữ session sống** khi đóng popup + **chấm đỏ** báo phản hồi mới.
- `src/features/chat/pages/ChatPage.jsx` + `planner.js` — ghim MAI trong sidebar /chat (sentinel `PLANNER_CONV_ID="planner"`).

Code quan trọng — widget **sở hữu** session nên stream không chết khi đóng popup, và bật chấm đỏ khi có trả lời lúc đang đóng:

```
const maiSession = useAIPlanSession(null); // hook nằm ở widget (luôn
mounted)
useEffect(() => { // tin nhắn tăng khi KHÔNG
xem MAI → chấm đỏ
    if (prev > 0 && len > prev && !viewingMai && last.role === "ASSISTANT")
setMaiUnread(true);
}, [maiSession.messages, viewingMai]);
```

`AIPlanPanel.jsx` — render Markdown để link Mravel bấm được, mở tab mới:

```
const html = marked.parse(content, {breaks:true, gfm:true})
    .replace(/<a /g, '<a target="_blank" rel="noopener noreferrer"
```

```
class="text-primary underline" ');
```

Luồng 9 — Bảo mật & phân quyền

File liên quan: `app/core/security.py` (giải mã JWT), `app/api/ai_plan.py` (`_assert_can_edit`), `app/core/errors.py` (`ForbiddenError` → 403).

Code quan trọng — gate quyền sửa, đọc `myRole` từ board:

```
async def _assert_can_edit(plan_client, bearer_token, plan_id):
    board = await plan_client.get_board(bearer_token, plan_id)
    role = str(board.get("myRole") or "").upper()
    if role not in {"OWNER", "EDITOR"}:
        raise ForbiddenError("Bạn chỉ có quyền XEM kế hoạch này nên không thể chỉnh sửa...")
    return board
```

Gate ở **3 chốt**: tạo phiên edit / `edit_stream` / `apply-edits`. plan-service vẫn là nơi enforce cuối (mọi mutation cần EDITOR). JWT chuyển tiếp tới plan-service khi ghi.

Bản đồ file (tra cứu nhanh)

<code>app/main.py</code>	App FastAPI, CORS, router, health
<code>app/config.py</code>	Toàn bộ env (LLM, Mongo, web search, max_tokens...)
<code>app/api/ai_plan.py</code>	Tất cả endpoint + dò plan id + <code>_assert_can_edit</code> + SSE
<code>app/api/dependencies.py</code>	DI: tạo singleton llm/catalog/plan/store/orchestrator
<code>app/agent/orchestrator.py</code>	Vòng lặp agent + 2 prompt (plan/edit) + primer (tiêm ngày)
<code>app/agent/tools.py</code>	Định nghĩa tool + ToolDispatcher + <code>mravel_url</code> + fallback
<code>app/agent/edits.py</code>	EditOperation + validate + board_summary + tóm tắt op
<code>app/llm/{base,openai_client,stub,factory}.py</code>	Lớp LLM (interface, impl, stub, chọn)
<code>app/services/session_store.py</code>	Mongo/File/Memory store + factory
<code>app/services/constraint_extractor.py</code>	Trích constraint (regex + LLM)
<code>app/services/draft_composer.py</code>	Dựng nháp tắt định (stub/fallback)
<code>app/services/approval_service.py</code>	Nháp → plan-service (tạo plan + card)
<code>app/services/edit_service.py</code>	Op chỉnh sửa → plan-service
<code>app/services/{geo,pricing}.py</code>	Khoảng cách / giá ước tính
<code>app/clients/{catalog,plan,web_search}_client.py</code>	REST upstream
<code>app/models/session.py</code>	PlanSession, Constraints, PlanDraft, ChatMessage
Frontend: <code>features/aiPlan/{hooks/useAIPlanSession.js, components/AIPlanPanel.jsx}</code>	
<code>features/chat/{components/FloatingChatWidget.jsx, pages/ChatPage.jsx, planner.js}</code>	

Cấu hình (đặt ở cuối theo yêu cầu)

`.env` (đã git-ignore, chứa secret) — copy từ `.env.example` rồi điền:

Biến	Ý nghĩa
<code>LLM_API_KEY</code>	Rỗng → chế độ stub (không gọi model). Có → chạy agent.
<code>LLM_BASE_URL</code> / <code>LLM_MODEL</code>	Đổi nhà cung cấp/model không cần sửa code (Chat Completions).
<code>LLM_EXTRACTOR_MODEL</code>	Model rẻ cho bước trích constraint.
<code>LLM_FALLBACK_MODELS</code>	Danh sách model thử khi model chính lỗi 429/402/5xx.
<code>LLM_MAX_TOKENS</code>	8000 — token đầu ra/lượt; kế hoạch dài cần lớn (2048 bị cắt).
<code>WEB_SEARCH_API_KEY</code>	Tavily; rỗng = tắt tìm web (giá → ước tính, gần nhãn).
<code>PUBLIC_WEB_BASE_URL</code>	Rỗng = link catalog tương đối (<code>/hotels/<slug></code>).
<code>SESSION_STORE_BACKEND</code>	<code>mongo</code> (khuyến nghị) <code>file</code> <code>memory</code> .
<code>MONGO_URI</code> / <code>MONGO_DB</code> / <code>SESSION_COLLECTION</code>	Atlas / <code>mravel_catalog</code> / <code>ai_plan_sessions</code> .
<code>JWT_SECRET</code>	Rỗng → giải mã JWT không verify chữ ký (plan-service verify lại khi ghi).

Chọn LLM (1 trong 2)

A. OpenRouter trực tiếp (đơn giản nhất):

```
LLM_API_KEY=sk-or-v1-<key>
LLM_BASE_URL=https://openrouter.ai/api/v1
LLM_MODEL=openai/gpt-oss-120b:free      # nói thẳng OpenRouter → bỏ tiền tố
"openrouter/"
```

B. 9Router (proxy cục bộ, cổng 20128, tự fallback):

```
npm install -g 9router && 9router      # thêm key OpenRouter trong dashboard
```

```
LLM_API_KEY=9router-local
LLM_BASE_URL=http://localhost:20128/v1
LLM_MODEL=openrouter/openai/gpt-oss-120b:free
```

Muốn kế hoạch **dày và chi tiết ổn định** → đổi `LLM_MODEL` sang model mạnh hơn (vd `google/gemini-2.5-flash`, `deepseek/deepseek-chat`, `gpt-4o-mini`). Model free `gpt-oss-120b` hay ra ít hoạt động.

MongoDB

Dùng cluster Atlas của dự án: điền `MONGO_URI` (giống chuỗi trong `catalog-service/.../application.yml`) và **whitelist IP** trong Atlas → Network Access. Không thì store sẽ log `falling back to file`. Dev thuần local có thể dùng `mongodb://localhost:27017` hoặc `SESSION_STORE_BACKEND=memory`.

Chạy & kiểm tra

```
cd Mravel_Backend\services\ai-plan-service
python -m venv .venv ; .\.venv\Scripts\Activate.ps1
pip install -e ".[dev]" # gồm pymongo
copy .env.example .env # điền theo bảng trên
python -m app.main # uvicorn :8092
```

- Log khởi động: `session store: mongo mravel_catalog/ai_plan_sessions`.
- Health: `http://localhost:8092/api/ai-plan/health`
- Test (không cần Mongo/LLM): `pytest -q` → 46 passed.
- Chạy kèm: `auth-service`, `catalog-service`, `plan-service`, `Mravel_Gateway`, MongoDB. Frontend tự hiện MAI trong khung chat nổi.